

# LEGACY LOCK

## PROBLEM STATEMENT:

Users need a secure way to store sensitive digital data and transfer it to nominees under specific conditions.

## Target Users:

Individuals managing digital assets  
(documents, passwords, medical records)

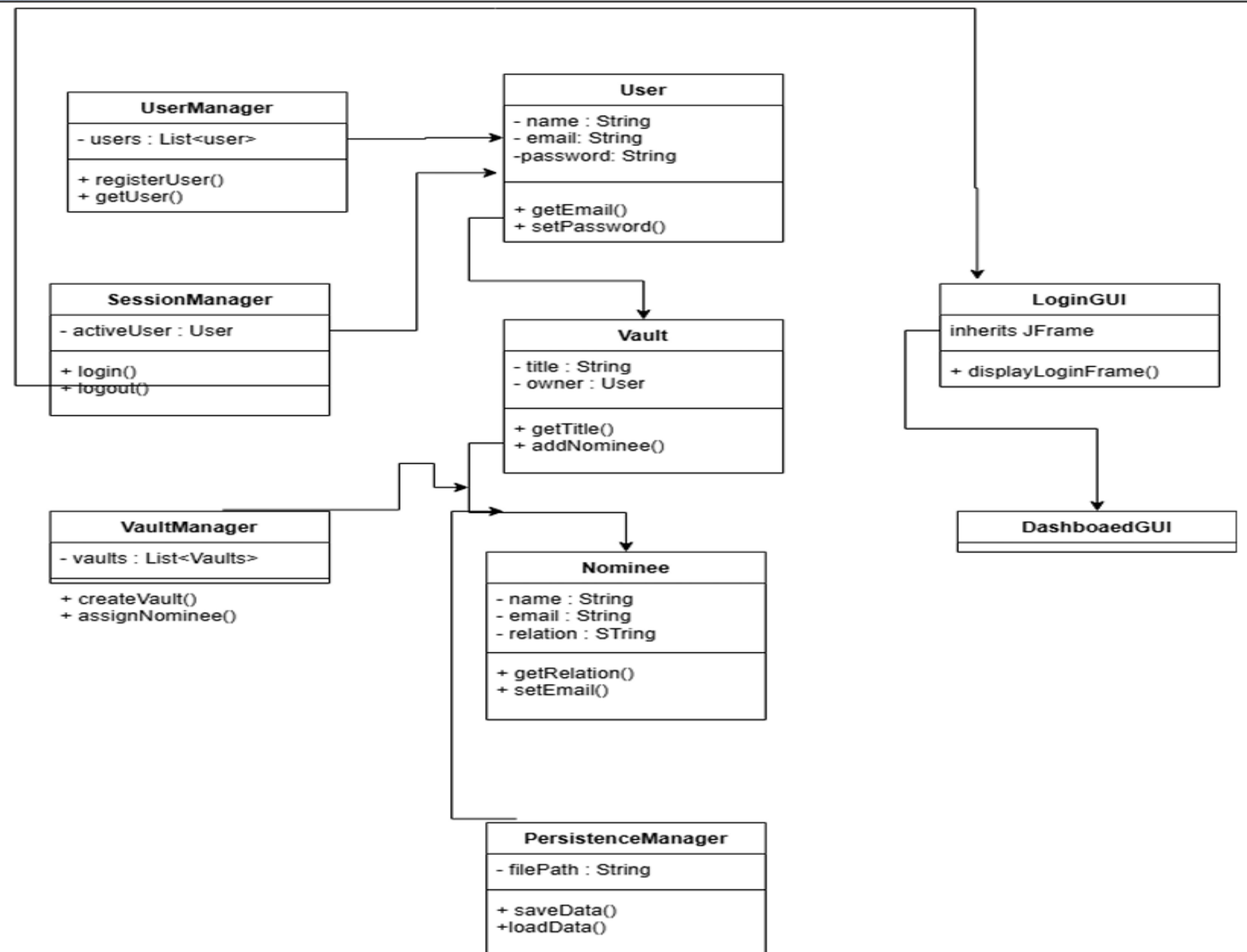
# 01

# ARCHITECTURE

## Package Structure:

src

- model
- manager
- auth
- gui
- security
- strategy
- monitor
- exception
- persistence
- main





# OOPS CONCEPT USED

- ◆ **Abstraction**

ReleaseCondition (strategy layer)

Defines **common behavior for vault release**

- ◆ **Inheritance**

VaultOwner, Nominee → extend User

DocumentItem, etc → extend VaultItem

- ◆ **Polymorphism**

shouldRelease():

Different behavior for each condition

Same method → different outcomes

- ◆ **Design Patterns**

**Singleton Pattern:** VaultManager, UserManager

**Strategy Pattern:** ReleaseCondition

02

# TECHNICAL HIGHLIGHT

# 03

## Feature: Vault Release System

Vault linked with:

Owner

Nominee

ReleaseCondition

VaultMonitor (thread):

Continuously checks conditions

Triggers release automatically

## This design:

Separates logic cleanly

Supports multiple conditions

Real-world automation system

Each vault has a condition, and a background thread continuously checks it. Once satisfied, the system automatically releases data to the nominee. This makes the system dynamic and realistic



# 04

## PROJECT DEMO

- User logs into the system using secure authentication.
- Files are uploaded to the vault and encrypted before storage.
- Nominee is assigned and linked to the vault.
- Vault is automatically released when the condition is satisfied.





# THANK YOU



WRAPPING UP AND  
PARTING THOUGHTS